

Number System

- Number is a Math value used for counting or measuring or labelling of.
- Number System is defined as a system of writing to express numbers. It is one the mathematical notation for representing no. of a given set by using digits.

Following are some N.S:- i) Decimal N.S.

ii) Binary N.S.

iii) Octal N.S.

iv) Hexadecimal N.S.

I) Decimal Number System- No. used in day-to-day life.

B

Base = 10 $\Rightarrow$ (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

$$\begin{array}{rcll} \text{eg- } 2586 & = & (2 \times 10^3) + (5 \times 10^2) + (8 \times 10^1) + (6 \times 10^0) \\ \begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \\ 10^3 \quad 10^2 \quad 10^1 \quad 10^0 \end{array} & = & 2000 + 500 + 80 + 6 \\ & & 2586 \end{array}$$

II) Binary N.S.- widely used in Computers. All inputs given to a comp. are 1st converted to B.N.S. b/c it understand only B.N.S..

Base = 2 $\Rightarrow$ (0, 1)

$$\begin{array}{rcll} \text{eg- } 10101 & = & (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\ \begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0 \end{array} & = & 16 + 0 + 4 + 0 + 1 \\ & = & (21)_{10} = (10101)_2 \end{array}$$

$$\begin{aligned} (10100011)_2 &= 128 + 0 + 32 + 0 + 0 + 0 + 2 + 1 \\ &= (163)_{10} \end{aligned}$$

III) Octal N.S. - Base $\Rightarrow 8$ (0, 1, 2, 3, 4, 5, 6, 7)

$$\text{eg:- } (2057)_8 = 1024 + 0 + 40 + 7$$

$$\begin{array}{cccc} 8^3 & 8^2 & 8^1 & 8^0 \\ 512 & 64 & 8 & 1 \end{array} = (1071)_{10}$$

$$\text{eg:- } (2671)_8 = 1024 + 384 + 56 + 1$$

$$= (1465)_{10}$$

$$\text{eg:- } (286)_8 = 128 + 64 + 6$$

invalid $\Rightarrow (198)_{10} \times$

IV) Hexadecimal N.S. - Base $\Rightarrow 16$ (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

16 Symbols or Character (single) $\left\{ \begin{array}{ll} 10 - A & 13 - D \\ 11 - B & 14 - E \\ 12 - C & 15 - F \end{array} \right\}$

$$(1AF)_{16} = 256 + 160 + 15$$

$$= (431)_{10}$$

$$(7CF)_{16} = 1792 + 192 + 15$$

$$= (1999)_{10}$$

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}
1	2	4	8	16	32	64	128	256	512	1024

2^{11}	2^{12}	8^0	8^1	8^2	8^3	8^4
2048	4096	1	8	64	512	4096

16^0	16^1	16^2	16^3	16^4
1	16	256	4096	65536



Converting from Decimal to Binary:-

$$\begin{array}{r}
 (25)_{10} \Rightarrow \begin{array}{r} 2 \ 25 \ 1 \\ 2 \ 12 \ 0 \\ 2 \ 6 \ 0 \\ 2 \ 3 \ 1 \\ 1 \end{array} \Rightarrow (11001)_2 \Rightarrow 16 + 8 + 0 + 0 + 1 \\
 = (25)_{10}
 \end{array}$$

$$\begin{array}{r}
 (42)_{10} \Rightarrow \begin{array}{r} 2 \ 42 \ 0 \\ 2 \ 21 \ 1 \\ 2 \ 10 \ 0 \\ 2 \ 5 \ 1 \\ 2 \ 2 \ 0 \\ 1 \end{array} \Rightarrow (101010)_2
 \end{array}$$

$$\begin{array}{r}
 (156)_{10} \Rightarrow \begin{array}{r} 2 \ 156 \ 0 \\ 2 \ 78 \ 0 \\ 2 \ 39 \ 1 \\ 2 \ 19 \ 1 \\ 2 \ 9 \ 1 \\ 2 \ 4 \ 0 \\ 2 \ 2 \ 0 \\ 1 \end{array} \Rightarrow (10011100)_2
 \end{array}$$

Decimal to Octal:-

$$\begin{array}{r}
 (952)_{10} = \begin{array}{r} 8 \ 952 \ 0 \\ 8 \ 119 \ 7 \\ 8 \ 14 \ 6 \\ 1 \end{array} \Rightarrow (1670)_8
 \end{array}$$

$$\begin{array}{r}
 (210)_{10} = \begin{array}{r} 8 \ 210 \ 2 \\ 8 \ 26 \ 2 \\ 3 \end{array} \Rightarrow (322)_8
 \end{array}$$

Converting Decimal to Hexadecimal:-

$$(428)_{10} = \begin{array}{r} 16 \ 428 \ 12 \\ 16 \ 26 \ 10 \\ 1 \end{array} \Rightarrow (1AC)_{16}$$

$$(188)_{10} = \begin{array}{r} 16 \ 188 \ 12 \\ 11 \end{array} \Rightarrow (BC)_{16}$$

Converting from a base other than 10 to
to base other than 10.

- 1) Convert the original no. to decimal no.
- 2) Convert the decimal no. to req no. (with new base).

$$\text{eg- } (101110)_2 = ()_8 \Rightarrow 32 + 8 + 4 + 2 \\ \Rightarrow (46)_{10}$$

$$\begin{array}{r} 8 \ 46 \ 6 \\ 5 \end{array} \Rightarrow (56)_8$$

$$(1AB)_{16} = 256 + 160 + 11 \\ = (427)_{10}$$

$$(11010011)_2 = 128 + 64 + 16 + 2 + 1 \\ = (211)_{10}$$

$$\begin{array}{r} 2 \ 427 \ 1 \\ 2 \ 213 \ 1 \\ 2 \ 106 \ 0 \\ 2 \ 53 \ 1 \\ 2 \ 26 \ 0 \\ 2 \ 13 \ 1 \\ 2 \ 6 \ 0 \\ 2 \ 3 \ 1 \\ 1 \end{array} \Rightarrow (110101011)_2$$

$$16 \ 211 \ 3 \Rightarrow (D3)_{16}$$

$$2 \ 53 \ 1$$

$$2 \ 26 \ 0$$

$$2 \ 13 \ 1$$

$$2 \ 6 \ 0$$

$$2 \ 3 \ 1$$

$$1$$

Shortcut from Binary to Octal

- 1) Divide Binary digit into group of 3 digits (starting from right).
- 2) Convert each group of 3 Binary Digit into one octal digits.

Decimal	HexaDecimal	Binary	Octal
0	0	0000	0
1	1	0001	1
2	2	0010	2
3	3	0011	3
4	4	0100	4
5	5	0101	5
6	6	0110	6
7	7	0111	7
8	8	1000	
9	9	1001	
10	A	1010	
11	B	1011	
12	C	1100	
13	D	1101	
14	E	1110	
15	F	1111	

$$\text{eg- } \underbrace{(101110)}_{5 \ 6}_2 \rightarrow (56)_8 \Rightarrow (56)_8$$

$$\underbrace{(1101010)}_{1 \ 5 \ 2}_2 \rightarrow (152)_8$$

Short cut for Octal to Binary:-

- 1) Convert each octal digit to 3 digit Binary Numbers.
- 2) Combine all resulting Binary groups into Single B. Numbers.

$$(562)_8 = (101\ 110\ 010)_2 \quad ; \quad 6751 = 110\ 111\ 101\ 001$$

Short for Binary to Hexadecimal:-

- 1) Convert each H.D. to 4 digit B. Numbers.
- 2) Combine all groups together

$$\begin{array}{c} \underline{(11101\ 1111)}_2 = (1DF)_{16} \\ \begin{array}{cc} 1 & D & F \\ 13 & 15 \end{array} \end{array}$$

$$\begin{array}{c} \underline{111\ 0010\ 1011}_2 = (72B)_{16} \\ \begin{array}{ccc} 7 & 2 & 11 \end{array} \end{array}$$

Short for Hexadecimal to Binary:-

$$(2AB)_{16} = (0010\ 1010\ 1011)_2$$

$$(ABC)_{16} = (1010\ 1011\ 1110)_2$$

Fractional no. Conversion:- [1st First Fraction to Decimal]

$$\begin{array}{c} \underline{(110\ .\ 101)}_2 \xrightarrow{-1/2\ -1/8} 6 + \frac{3}{8} \Rightarrow 6 + 0.625 \\ \begin{array}{cc} 421 & -1/4 \\ 6 & 2+0+1/8 \end{array} \end{array} \Rightarrow \boxed{6.625}_{10}$$

$$(127.54)_8 = 64 + 16 + 7 + \frac{5}{8} + \frac{4}{64} \Rightarrow \boxed{127.6875}_{10}$$

$$2B.C4 \rightarrow 32 + 11 + \frac{1^2}{16} + \frac{4}{256} \Rightarrow (43.765625)_{10}$$

$$(0.6875)_{10} = (?)_2$$

Base

	Fraction	integer
$0.6875 \times 2 =$	0.3750	1
$0.3750 \times 2 =$	0.7500	0
$0.7500 \times 2 =$	0.5000	1
$0.5000 \times 2 =$	0.0000	1

$$= (0.1011)_2$$

Binary Arithmetic:-

Addⁿ- $0+0=0$
 $0+1=1$
 $1+0=1$
 $1+1=0$ (and 1 carry to next higher column.)

$$\begin{array}{r} 10011 \\ 01001 \\ \hline 11100 \end{array}$$

$$\begin{array}{r} 10001 \\ 11101 \\ \hline 101110 \end{array}$$

$$\begin{array}{r} 101101 \\ 11001 \\ \hline 1000110 \end{array}$$

Subⁿ- $0-0=0$
 $1-0=1$
 $1-1=0$
 $0-0=1$ (Borrow from next column.)

$$\begin{array}{r} 1011011 \\ 10010 \\ \hline 1001001 \end{array}$$

$$\begin{array}{r} 101101 \\ 100111 \\ \hline 000110 \end{array}$$

$$\begin{array}{r} 1000101 \\ 101100 \\ \hline 0001001 \end{array}$$

mult

$0 * 0 = 0$	1101	1010
$0 * 1 = 0$	100	101
$1 * 0 = 0$	0000	1010
$1 * 1 = 1$	$0000x$	$0000x$
	$1101xx$	$1010xx$
	<u><u>110100</u></u>	<u><u>110010</u></u>

Div

$0 / 0 = \text{undefined}$

$0 / 1 = 0$

$1 / 0 = \text{undefined}$

$1 / 1 = 1$

BCD $\rightarrow$ Binary Coded Decimal $\rightarrow$ It is a method of representing Decimal no. in Binary, where each Decimal Digit is stored separately using 4 bit Binary code. Each D. Digit is converted into its corresponding 4 bit B-code. Every digit is encoded separately instead of converting the entire D.no into Binary.

D. No	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

eg- $57 \rightarrow (01010111)_{BCD}$

$\downarrow$ $\downarrow$
 0101 0111 BCD

$$1 \text{ Byte} = 8 \text{ bit}$$

$$1 \text{ nibble} = 4 \text{ bit} = \frac{1}{2} \text{ Byte}$$

→ Packed BCD: It stores 2 Digit in a single 8-bit; each 4 bit (nibble) represents 1 D. digit.

$$\begin{array}{l} \text{eg:- } 9 \rightarrow 1001 \\ \quad \quad 3 \rightarrow 0011 \end{array} \rightarrow (93)_{10} \rightarrow (10010011)_{\text{BCD}}$$

→ Unpacked BCD: It stores 1 D. digit per byte.

$$\begin{array}{l} \text{eg:- } 9 \rightarrow 00001001 \\ \quad \quad 3 \rightarrow 00000011 \end{array} \rightarrow (93)_{10} \rightarrow (00001001 \ 00000011)_{\text{BCD}}$$

→ BCD-Addition: To add 2 BCD no., add corresponding 4-bit groups as normal binary numbers. If the sum of any 4-bit group is > than 1001 (9); then 0110 (6) is added as a correction to obtain a valid BCD.

$$\begin{array}{r} \text{eg:- } 0010 \\ \quad 0101 \\ \hline 0111 \end{array} \quad \begin{array}{r} 23 \\ 45 \\ \hline 68 \end{array} \quad \begin{array}{r} 0010 \ 0011 \\ 0100 \ 0101 \\ \hline 0110 \ 1000 \\ \quad (6) \quad (8) \end{array}$$

$$\begin{array}{r} 45 \ 0100 \ 0101 \\ 37 \ 0011 \ 0111 \\ \hline 0111 \ 1100 \\ \quad 0110 \\ \hline \end{array}$$

$$82 \rightarrow \underline{\underline{1000 \ 0010}}$$

→ BCD - Subtraction: To subtract 2 BCD nos,
 Subtract the corresponding BCD Digits, if
 the result is not a valid BCD Digit,
 a correction factor (0110) (6) is applied.

$$\begin{array}{r}
 47 \quad 0100 \quad 0111 \\
 \underline{23 \quad 0010 \quad 0011} \\
 24 \quad \underline{\underline{00 \quad 0 \quad 0100}}
 \end{array}$$

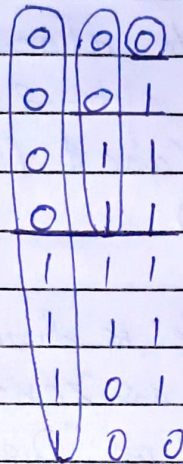
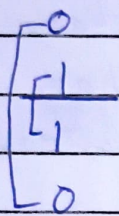
→ XS-3 Code: It is a non-weighted code,
 used to express decimal nos.

This code is derived from BCD code by adding
 $(0011)_2$ or $(3)_{10}$ to each BCD code.

0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100



→ Gray Code:- It is a non weighted code, and its not arithmetic code that means there are no specific weight assigned to the bit position, it has a very special feature that has only one bit change, each time the decimal no. is increment as only 1 bit change at a time, It is also called as unit distance code. Used in K-map.



→ ASCII Code:- Bits can also be coded to represent Alphabets, Numbers & Sp. character.

→ American Std. Code for Info. Interchange.

→ $2^7 = 128$ Char.

→ "A" is → 65 ; "Z" → 90

"a" is → 97 ; "z" → 122

"0" is → 48 ; "9" → 57.

→ EBCDIC Code:-

- ↳ Extended Binary Coded Decimal Interchange Code.
- ⇒ An 8-bit char. encoding system developed by IBM in 1963-64, for their mainframe & mid-range comp. systems.
- ⇒ It translates human readable char. like letters, no. & sp. symbols into B.Code, so that large comp. processors can understand and store them.
- ⇒ It uses 1 byte to represent a single char.
- ⇒ It can represent upto 256 Unicoeue char. (Including Upper Case & (2⁸) Lower Case letters, nos, & Sp. char.)
- ⇒ Each 8-bit code is divided into 2 equal parts:-
 - 1) 1st 4 bits act as "zone",
 - 2) Las 4 bits act as "Digit".

Boolean Algebra:- It deals with the B.no. System, the variable used in the Boolean eqⁿ may assume only 2 possible value (0 & 1).

→ Logical Addition:- Symbol "+" is used for L.A. operator:

→ It is called "OR" operator.

→ Truth table for the Logical "OR" operator is:-

Input		Output
A + B	=	C
0 + 0	=	0
0 + 1	=	1
1 + 0	=	1
1 + 1	=	1

⇒ Logical Multiplier:- Symbol "." used for multiplication operator.

→ Also called "AND" operator.

→ Truth table for the Logical "AND" operator is:-

Input		Output
A . B	=	C
0 . 0	=	0
0 . 1	=	0
1 . 0	=	0
1 . 1	=	1